Text Similarity and Embedding

Text Similarity

- The similarity measure is the measure of how much a like two data objects are.
- A similarity measure is a data mining or machine learning context is a distance with dimensions representing features of the objects.
  - If the distance is small, the features are having a high degree of similarity.
  - Whereas a large distance will be a low degree of similarity.
- The similarity is subjective and is highly dependent on the domain and application.

Similarity and normalization

- For example, two fruits are similar because of color or size or taste. Special care should be taken when calculating distance across **dimensions/features** that are unrelated.
- The relative values of each element must be **normalized (to be in range from 0 to 1)**, or one feature could end up dominating the distance calculation.
- Generally, **similarity are measured in the range 0 to 1 [0,1].**
- In the machine learning world, this score in the range of [0, 1] is called the similarity score.
- **Two main consideration of similarity:**
  - Similarity = 1 if X = Y (Where X, Y are two objects)
  - Similarity = 0 if X ≠ Y

Example usage of Similarity

This similarity is the very basic building block for activities such as

- Recommendation engines,
- clustering,
- Different classification problems,
- Email spam or ham classification problems

Five Common Similarity measures

1. Euclidean distance
2. Manhattan distance
3. Minkowski distance
4. Cosine Similarity
5. Jaccard Similarity

https://dataaspirant.com/five-most-popular-similarity-measures-implementation-in-python/

Lexical Similarity and Semantic Similarity-1

- Lexical and semantic similarity are two different ways to measure how alike two pieces of text are.
- The key difference is that **lexical similarity** focuses on the words themselves, while **semantic similarity** focuses on their meaning.

Lexical Similarity

Lexical similarity measures the resemblance of two texts at the word or character level. It's a surface-level comparison ظاهرية that looks for shared words, word stems, or character sequences.
It's a great way to identify if texts are paraphrased using similar vocabulary or if two languages share common roots.

How it works:

- It counts the number of identical words between two texts.
- Common methods include the **Jaccard similarity coefficient**, which is the number of shared words divided by the total number of unique words in both texts, and **Levenshtein distance**, which counts the minimum number of single-character edits needed to change one word into another.

Example:

- Sentence A: "The **cat** sat on the **mat**."
- Sentence B: "The **dog** sat on the **mat**."
- Lexical similarity is high because they share "the," "sat," "on," and "mat."

Lexical Similarity and Semantic Similarity-2

Semantic Similarity

Semantic similarity measures how similar two texts are based on their meaning, regardless of the words used.
It's a deeper, more conceptual comparison that understands the context and relationships between words.
This is a core concept in Natural Language Processing (NLP).

How it works:

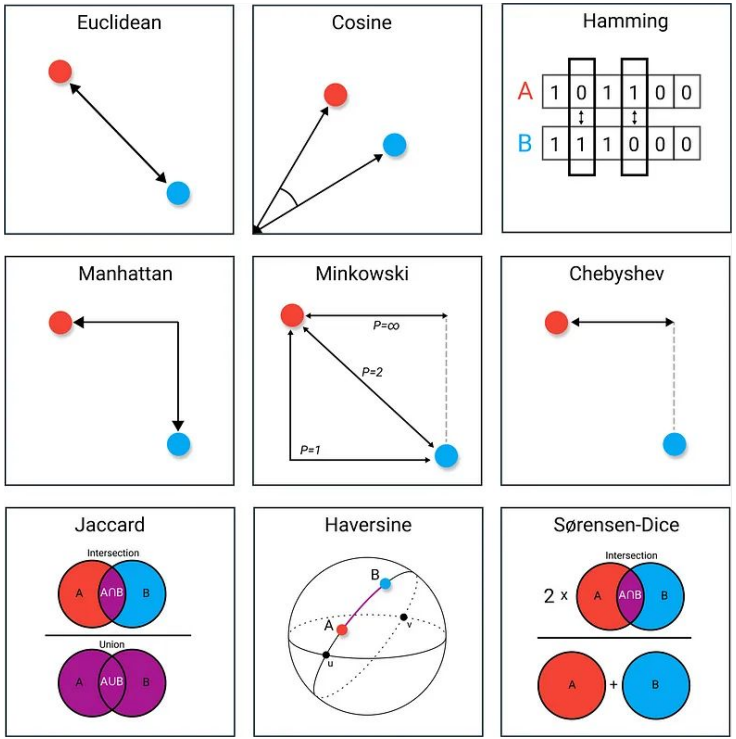
- It relies on **word embeddings** or vector representations of words and sentences. These vectors are trained on large corpora of text, so that words with similar meanings are located closer to each other in a multi-dimensional space.
- The similarity is calculated by measuring the "distance" between these vectors, often using **cosine similarity**.

Example:

- Sentence A: "I'm a big fan of coding."
- Sentence B: "Programming is my passion."
- Lexical similarity is low because there are no shared words, but semantic similarity is very high because "coding" and "programming" have similar meanings, as do "fan of" and "passion."

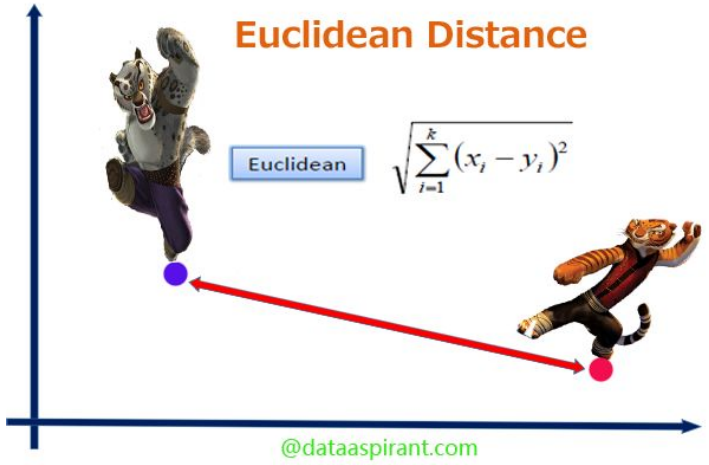
Distance (similarity) Measures in Data Science

للاطلاع فقط



Detailed reading:
https://towardsdatascience.com/9-distance-measures-in-data-science-918109d069fa

1- Euclidean distance



Euclidean distance is the most common use of distance measure.

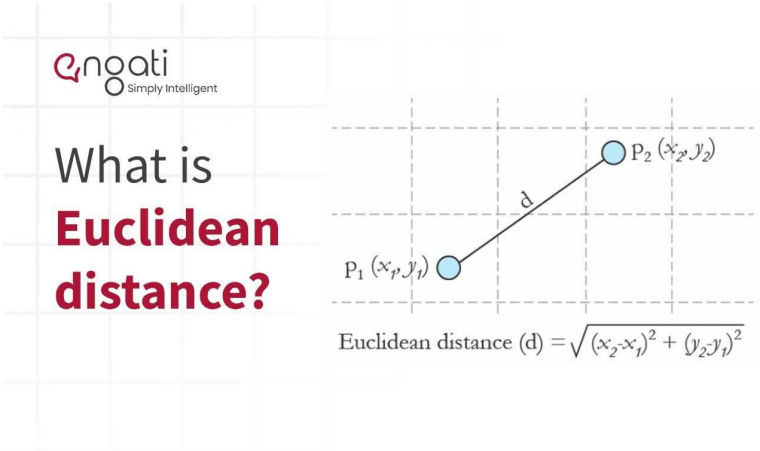
In most cases when people say about distance, they will refer to Euclidean distance.

Euclidean distance is also known as simply **distance**. **(straight line** distance في المسافة خط مستقيم)

When data is dense or **continuous**, this is the best proximity measure.

The Euclidean distance between two points is the length of the path connecting them.

The **Pythagorean theorem** gives this distance between two points.



Euclidean distance implementation in python:

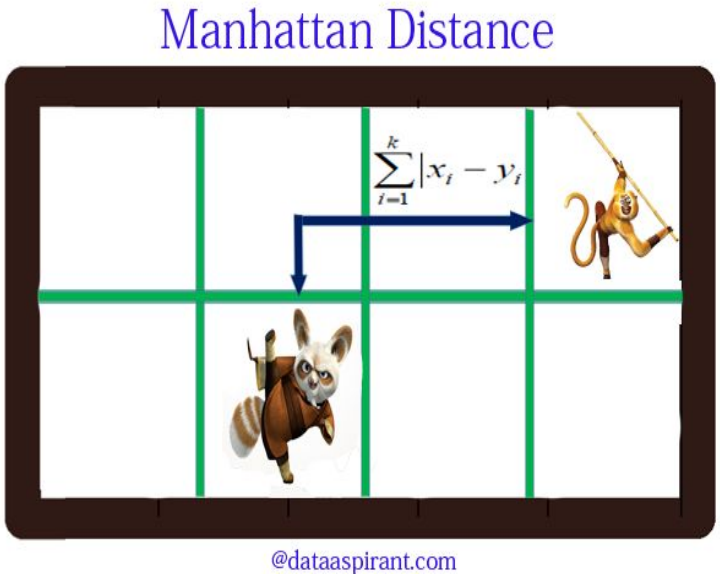
```
#!/usr/bin/env python

from math import*

def euclidean_distance(x,y):

    return sqrt(sum(pow(a-b,2) for a, b in zip(x, y)))

print euclidean_distance([0,3,4,5],[7,6,3,-1])
```

Script Output:

```
9.74679434481
[Finished in 0.0s]
```

2- Manhattan distance:

- Manhattan distance is a metric in which the distance between two points is calculated as the sum of the **absolute differences** of their Cartesian coordinates.
- In a simple way of saying it is the total sum of the difference between the x-coordinates and y-coordinates.
- Suppose we have two points A and B. If we want to find the Manhattan distance between them, just we have, to sum up, the absolute x-axis and y-axis variation. This means we have to find how these two points A and B are varying in X-axis and Y-axis. In a more mathematical way of saying Manhattan distance between two points measured along axes at right angles.



- In a plane with p1 at (x1, y1) and p2 at (x2, y2).
  - Manhattan distance = |x1 – x2| + |y1 – y2|

Manhattan distance

- The Manhattan distance, also known as city blocks and taxicab, between two vectors is equal to the one-norm of the distance between the vectors

$$d_1\left(u,v\right) = \sum_{i \in I_{uv}} \left(\left| r_{vi} - r_{ui} \right| \right) \tag{24}$$

where I_{uv} denotes the set of items commonly rated by both u and v, r_{ui} and r_{vi} denote the rating of the user u and v,

https://www.sciencedirect.com/science/article/pii/S1319157821002652

Manhattan distance example

- Suppose 2 vectors
  - row1 = [10, 20, 15, 10, 5]
  - row2 = [12, 24, 18, 8, 7]
- Then their Manhattan distance =

|10-12| +
|20-24| +
|15-18| +
|10-8|+
|5-7| = 13

/https://machinelearningmastery.com/distance-measures-for-machine-learning

Manhattan distance implementation in python:

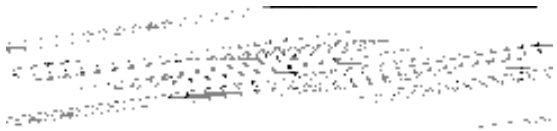
```
#!/usr/bin/env python

from math import*
def manhattan_distance(x,y):
    return sum(abs(a-b) for a,b in zip(x,y))
print manhattan_distance([10,20,10],[10,20,20])
```

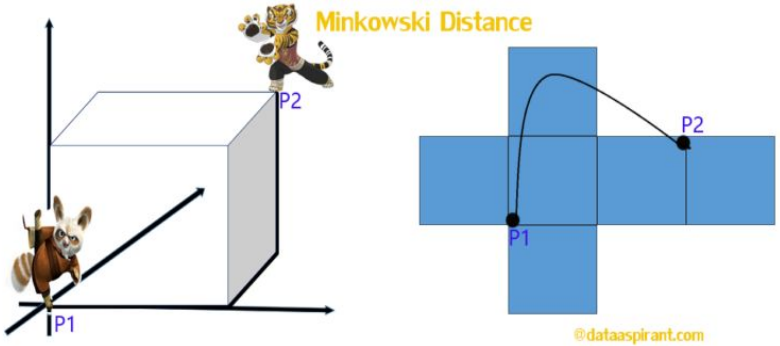
Script Output:

```
10
[Finished in 0.0s]
```

3- Minkowski distance

The Minkowski distance is a generalized metric form of **Euclidean distance** and Manhattan distance.





In the equation:

- **d^MKD** is the Minkowski distance between the data record i and j,
- k the index of a variable,
- n the total number of variables y
- and λ the order of the Minkowski metric.
- Although it is defined for any λ > 0, it is rarely used for values other than 1, 2, and ∞.

The way distances are measured by the Minkowski metric of different orders between two objects with three variables (In the image is displayed in a coordinate system with x, y, z-axes).

Synonyms of Minkowski

Different names for the Minkowski distance or Minkowski metric arise from the order:

- $\lambda = 1$ is the Manhattan distance.
- $\lambda = 2$ is the Euclidean distance.
- $\lambda = \infty$ is the Chebyshev distance.

Minkowski distance implementation in python

```
#!/usr/bin/env python


from math import*
from decimal import Decimal
def nth_root(value, n_root):
    root_value = 1/float(n_root)
    return round (Decimal(value) ** Decimal(root_value),3)
def minkowski_distance(x,y,p_value):
    return nth_root(sum(pow(abs(a-b),p_value) for a,b in zip(x, y)),p_value)
print minkowski_distance([0,3,4,5],[7,6,3,-1],3)
```

Script Output:

```
8.373
[Finished in 0.0s]
```

4- Cosine Similarity

The cosine similarity metric finds the **normalized dot product** of the two attributes.

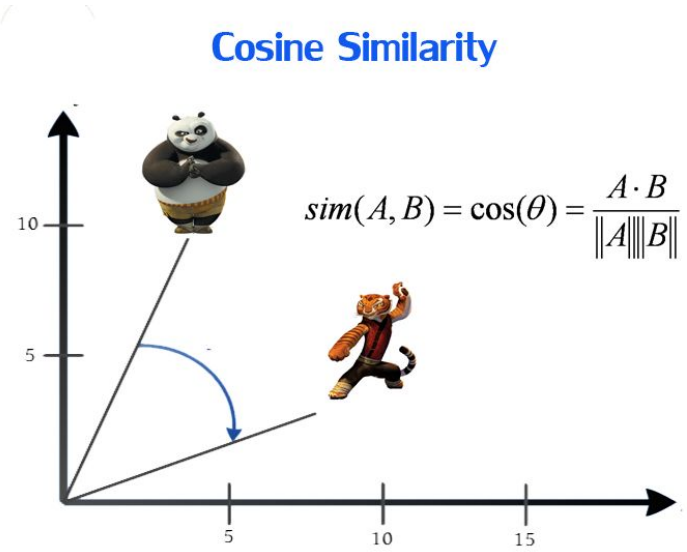
By determining the cosine similarity, we would effectively try to find the cosine of the angle between the two objects.

The cosine of **0° is 1**, and it is **less than 1** for any other angle.

It is thus a judgment of orientation (angle) and not magnitude (length).

- Two vectors with the same orientation have a cosine similarity of 1,
- two vectors at 90° have a similarity of 0.
- Whereas two vectors diametrically opposed (180°) having a similarity of -1, independent of their magnitude.

Cosine similarity is particularly used in positive space, where the outcome is neatly bounded in [0,1]. One of the reasons for the popularity of cosine similarity is that it is very efficient to evaluate, especially for sparse vectors.



Cosine similarity implementation in python

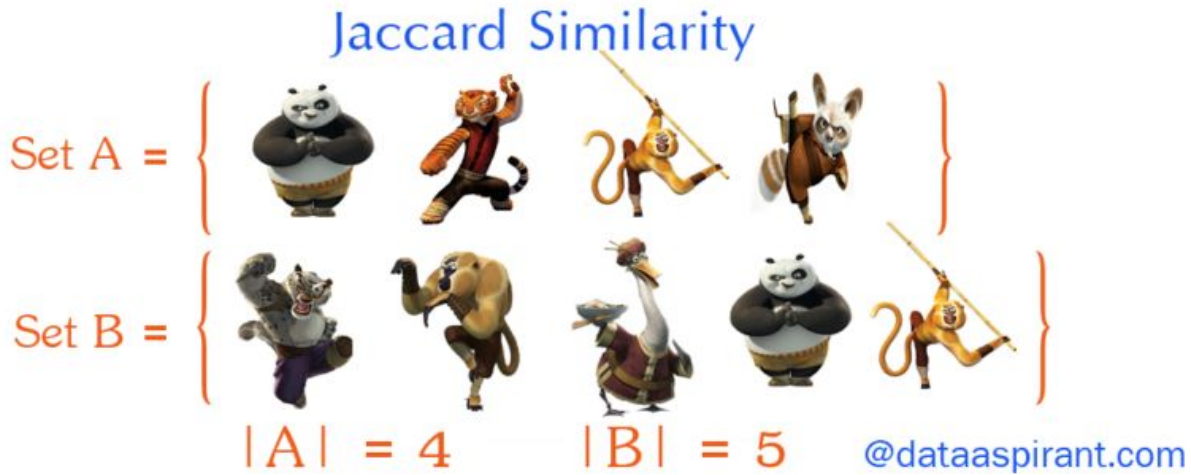
```
#!/usr/bin/env python
from math import*


def square_rooted(x):
    return round(sqrt(sum([a*a for a in x])),3)


def cosine_similarity(x,y):
    numerator = sum(a*b for a,b in zip(x,y))
   denominator = square_rooted(x)*square_rooted(y)
   return round(numerator/float(denominator),3)
print cosine_similarity([3, 45, 7, 2], [2, 54, 13, 15])
```

Script Output:

```
0.972
[Finished in 0.1s]
```

5- Jaccard similarity

- The previous metrics are used to find the similarity between objects. where the objects are points or vectors.
- When we consider Jaccard similarity these objects will be sets. مجموعات



Sets & Set Operations

Sets: A set is (unordered) collection of objects {a,b,c}. we use the notation as elements separated by commas inside curly brackets { }. They are unordered so {a,b} = { b,a }.

Cardinality

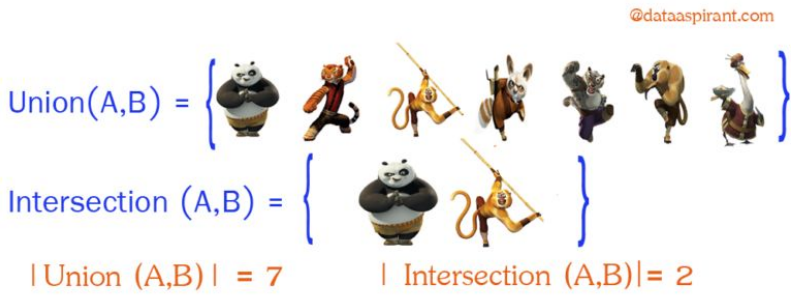
The cardinality of A denoted by **|A|** which counts how many elements are in A.

Intersection

The intersection between two sets A and B is denoted **A ∩ B** and reveals all items which are in both sets A, B.

Union

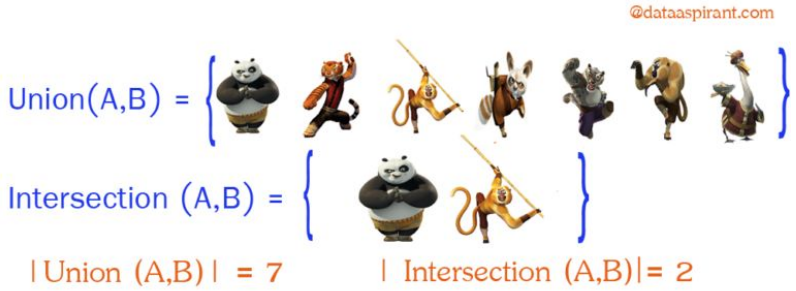
The union between two sets A and B is denoted **A ∪ B** and reveals all items which are in either set.



Example

Now going back to Jaccard similarity. The Jaccard similarity measures the similarity between finite sample sets and is defined as the cardinality of the intersection of sets divided by the cardinality of the union of the sample sets.

Suppose you want to find Jaccard similarity between two sets A and B it is the ration of the cardinality of **A ∩ B and A ∪ B**



Jaccard Similarity J (A,B) = | Intersection (A,B) | / | Union (A,B) |

= 2 / 7

= 0.286

Jaccard similarity implementation in Python

```
#!/usr/bin/env python


from math import*
def jaccard_similarity(x,y):
    intersection_cardinality = len(set.intersection(*[set(x), set(y)]))
    union_cardinality = len(set.union(*[set(x), set(y)]))
    return intersection_cardinality/float(union_cardinality)

print jaccard_similarity([0,1,2,5,6],[0,2,3,5,7,9])
```

Script Output:

```
0.375
[Finished in 0.0s]
```

Phonetic transformation algorithms

- Phonetic transformation algorithms can be used to identify words that sound similar, even if they are spelled differently (e.g. "Stephen" vs "Steven").
- These algorithms to give another type of fuzzy match and are often generated in the Feature Engineering step of record linkage.
- **Algorithms**
  - Soundex
  - Metaphone
  - Double Metaphone

https://moj-analytical-services.github.io/splink/topic_guides/comparisons/phonetic.html

What is Soundex?

Soundex is a phonetic algorithm for encoding and indexing names base on sound. The goal is for homophones to be encoded to the same representation so that they can be matched despite minor differences in spelling. The code consists of the first letter of the name, followed by 3 digits representing the first three phonetic sounds found in the name. For example, Sherman, Sharman, Shurman, Shearman, Shireman, Scherman, Schurman and Sirman are all spelling variations of name. With the Soundex system, they all have the Soundex code S-655.

Spell Checking

- Important part of query processing
  - 10-15% of all web queries have spelling errors
- Errors include typical word processing errors but also many other types, e.g.

poiner sisters
brimingham news
catamarn sailing
hair extenssions
marshmellow world
miniture golf courses
psyhics
home doceration

realstateisting.bc.com
akia 1080i manunal
ultimatwarcade
mainscourcebank
dellottitouche

Spell Checking

- Basic approach: suggest corrections for words not found in *spelling dictionary*
- Suggestions found by comparing word to words in dictionary using similarity measure
- Most common similarity measure is *edit distance*
  - number of operations required to transform one word into the other

Edit Distance

- *Damerau-Levenshtein* distance
  - counts the minimum number of insertions, deletions, substitutions, or transpositions of single characters required
  - e.g., Damerau-Levenshtein distance 1

$$\text{doceration} \rightarrow \text{deceration}$$
$$\text{deceration} \rightarrow \text{decoration}$$

  - distance 2

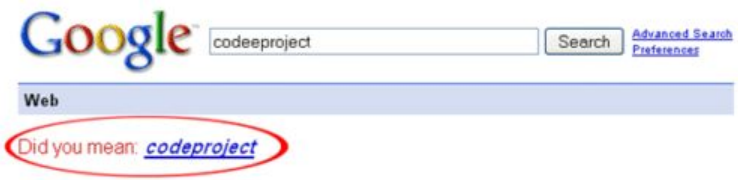
$$\text{extenssions} \rightarrow \text{extensions} (\text{insertion error})$$
$$\text{poiner} \rightarrow \text{pointer} \ (\text{deletion error})$$
$$\text{marshmellow} \rightarrow \text{marshmallow} \ (\text{substitution error})$$
$$\text{brimingham} \rightarrow \text{birmingham} \ (\text{transposition error})$$

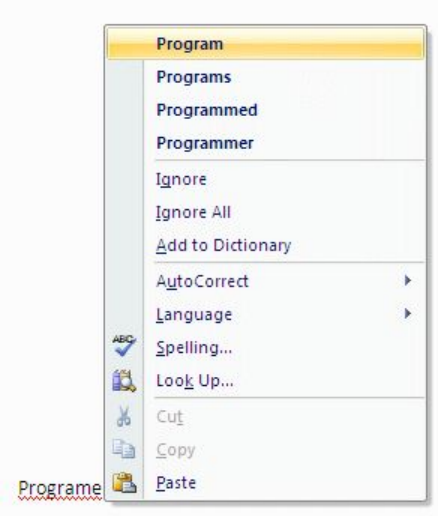
Edit Distance

- Number of techniques used to speed up calculation of edit distances
  - restrict to words starting with same character
  - restrict to words of same or similar length
  - restrict to words that sound the same
- Last option uses a *phonetic code* to group words
  - e.g. Soundex

Soundex introduction

- Soundex is a phonetic algorithm for indexing names by sound. Many applications use algorithms like this to add fantastic features like the Google Spelling Corrections and MS Word Autocorrect.





https://www.codeproject.com/Articles/26880/Arabic-Soundex

Soundex Code

1. Keep the first letter (in upper case).
2. Replace these letters with hyphens: a,e,i,o,u,y,h,w.
3. Replace the other letters by numbers as follows:

   1: b,f,p,v
   2: c,g,j,k,q,s,x,z
   3: d,t
   4: l
   5: m,n
   6: r

4. Delete adjacent repeats of a number.
5. Delete the hyphens.
6. Keep the first three numbers or pad out with zeros.

extenssions → E235; extensions → E235
marshmellow → M625; marshmallow → M625
brimingham → B655; birmingham → B655
poiner → P560; pointer → P536

Example

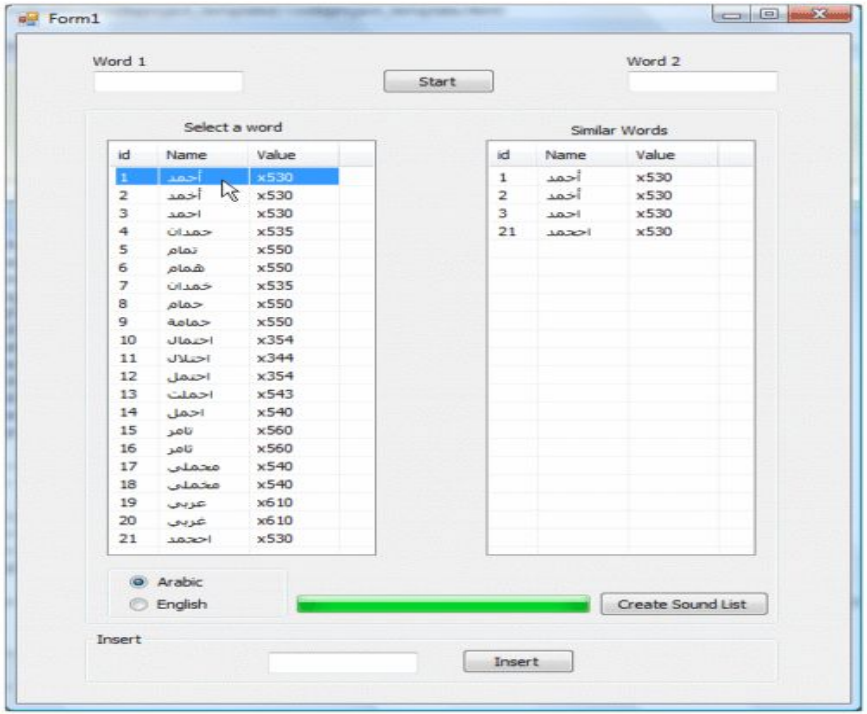
Example

You can test out the Soundex transformation between two strings through the phonetics package.

```
import phonetics

print(phonetics.soundex("Smith"), phonetics.soundex("Smyth"))
```

```
S5030 S5030
```

Arabic Soundex



https://www.codeproject.com/Articles/26880/Arabic-Soundex

Arabic Soundex-2

- The Soundex algorithm stands on grouping similar sounding letters depending on special sounding features, as follows:

Soundex Code	المحارف	المجموعة
1	b, f, p, v	شفهي
2	c, g, j, k, q, s, x, z	حروف حلقية و حروف صفير
3	d, t	حرف نطعي (ملفوظ بوضع اللسان على مؤخر الاسنان الامامية العليا)
4	l	حرف صامت ملفوظ بلطف طويل
5	m, n	حرف يلفظ من الانف
6	r	حرف صامت ملفوظ بلطف قصير

https://www.codeproject.com/Articles/26880/Arabic-Soundex

Arabic Soundex-3

1. Hold the first letter.
2. Replace the characters ي,و,ش, غ, ع, ح, آ, إ, أ, ا)) with the value 0.
3. Replace the characters (ب) ,ف with the value 1.
4. Replace the **characters ((** ك ,ق ,ظ ,ص ,س ,ز ,ج ,خ with the value 2.
5. Replace the characters **((** ط,ض,ذ,د,ث, ت with the value 3.
6. Replace the character **((** ل with the value 4.
7. Replace the characters **(**ن**)** ,م with the value 5.
8. Replace the character (ر**)** with the value 6.
9. Remove the (ا, أ, آ, إ) characters from the **beginning** of the word if found, since they added more confusion.
10. Ignore the first character handling: in English language, it is important to handle the first character, but not always in Arabic
11. Update the character sound categories by removing or adding some characters.

https://www.codeproject.com/Articles/26880/Arabic-Soundex

Soundex Limitation

There are some limitation when using soundex, that was:

- **Unrelated names may be grouped together.** Sometimes names that do not appear to be related show up together on a Soundex index. Ignore clearly unrelated names. For example, if you were looking for **Wilkins**, you may also find under the same Soundex code, W425, the name **Walakynowski**.
- **Related names may not be grouped together.** Sometimes names that are obviously related do not come together in the same Soundex index group. For example, **Clausen** is under C425 and **Klausen** under K425.
- **Soundex also has trouble discriminating long names.** Sometimes name with 3 or more phonetic sounds can hanve same soundex value. For example, **Mahulena** is under M450 and **Mulyama** under M450.

Using Soundex Database

When using database soundex is the most widely known of phonetic algorithms. In some database like PostgreSQL, MySQL, SQLite, MS SQL Server, And Oracle Soundex become standard feature that we can use when searching name. Here are some examples of how to use soundex on some of the databases that have been mentioned:

PostgreSQL

```
CREATE EXTENSION IF NOT EXISTS fuzzystrmatch;


SELECT *

FROM artists

WHERE SOUNDEX(name) = SOUNDEX('Damian Hurst');
```

MySQL, SQLite, MS SQL Server, Oracle

```
SELECT *
FROM artists

WHERE SOUNDEX(name) = SOUNDEX('Damian Hurst');
```

2- Metaphone

Metaphone is an improved version of the Soundex algorithm that was developed to handle a wider range of words and languages.

The Metaphone algorithm assigns a code to a word based on its phonetic pronunciation, but it takes into account the sound of the **entire word**, rather than just its first letter and consonants.

The Metaphone algorithm works by applying a set of rules to the word's pronunciation, such as converting the "TH" sound to a "T" sound, or removing silent letters.

The resulting code is a variable-length string of letters that represents the word's pronunciation.

Metaphone algorithm

The Metaphone algorithm works by following these steps:

1. Convert the word to uppercase and remove all non-alphabetic characters.
2. Apply a set of pronunciation rules to the word, such as:
    1. Convert the letters "C" and "K" to "K"
    2. Convert the letters "PH" to "F"
    3. Convert the letters "W" and "H" to nothing if they are not at the beginning of the word
3. Apply a set of replacement rules to the resulting word, such as:
    1. Replace the letter "G" with "J" if it is followed by an "E", "I", or "Y"
    2. Replace the letter "C" with "S" if it is followed by an "E", "I", or "Y"
    3. Replace the letter "X" with "KS"
4. If the resulting word ends with "S", remove it.
5. If the resulting word ends with "ED", "ING", or "ES", remove it.
6. If the resulting word starts with "KN", "GN", "PN", "AE", "WR", or "WH", remove the first letter.
7. If the resulting word starts with a vowel, retain the first letter.
8. Retain the first four characters of the resulting word, or pad it with zeros if it has fewer than four characters.

Example

Example

You can test out the Metaphone transformation between two strings through the phonetics package.

```
import phonetics

print(phonetics.metaphone("Smith"), phonetics.metaphone("Smyth"))
```

```
SM0 SM0
```

3- Double Metaphone

- Double Metaphone is an extension of the Metaphone algorithm that generates two codes for each word:
  - one for the primary pronunciation
  - and one for an alternate pronunciation.
- The Double Metaphone algorithm is designed to handle a wide range of languages and dialects اللهجات المختلفة لنفس اللغة, and it is more accurate than the original Metaphone algorithm.
- The Double Metaphone algorithm works by applying a set of rules to the word's pronunciation, similar to the Metaphone algorithm, but it generates two codes for each word.
- The primary code is the most likely pronunciation of the word, while the alternate code represents a less common pronunciation.

Algorithm (standard double metaphone)

The Double Metaphone algorithm works by following these steps:

1. Convert the word to uppercase and remove all non-alphabetic characters.
2. Apply a set of pronunciation rules to the word, such as:
    a. Convert the letters "C" and "K" to "K"
    b. Convert the letters "PH" to "F"
    c. Convert the letters "W" and "H" to nothing if they are not at the beginning of the word
3. Apply a set of replacement rules to the resulting word, such as:
    a. Replace the letter "G" with "J" if it is followed by an "E", "I", or "Y"
    b. Replace the letter "C" with "S" if it is followed by an "E", "I", or "Y"
    c. Replace the letter "X" with "KS"
4. If the resulting word ends with "S", remove it.
5. If the resulting word ends with "ED", "ING", or "ES", remove it.
6. If the resulting word starts with "KN", "GN", "PN", "AE", "WR", or "WH", remove the first letter.
7. If the resulting word starts with "X", "Z", "GN", or "KN", retain the first two characters.
8. Apply a second set of rules to the resulting word to generate an alternative code.
9. Return the primary and alternative codes as a tuple.

https://moj-analytical-services.github.io/splink/topic_guides/comparisons/phonetic.html#__tabbed_1_1

Algorithm (alternate double metaphone)

The Alternative Double Metaphone algorithm takes into account different contexts in the word and is generated by following these steps:

1. Apply a set of prefix rules, such as:
    a. Convert the letter "G" at the beginning of the word to "K" if it is followed by "N", "NED", or "NER"
    b. Convert the letter "A" at the beginning of the word to "E" if it is followed by "SCH"
2. Apply a set of suffix rules, such as:
    a. Convert the letters "E" and "I" at the end of the word to "Y"
    b. Convert the letters "S" and "Z" at the end of the word to "X"
    c. Remove the letter "D" at the end of the word if it is preceded by "N"
3. Apply a set of replacement rules, such as:
    a. Replace the letter "C" with "X" if it is followed by "IA" or "H"
    b. Replace the letter "T" with "X" if it is followed by "IA" or "CH"
4. Retain the first four characters of the resulting word, or pad it with zeros if it has fewer than four characters.
5. If the resulting word starts with "X", "Z", "GN", or "KN", retain the first two characters.
6. Return the alternative code.

https://moj-analytical-services.github.io/splink/topic_guides/comparisons/phonetic.html#__tabbed_1_1

Example

Example

You can test out the Metaphone transformation between two strings through the phonetics package.

```
import phonetics

print(phonetics.dmetaphone("Smith"), phonetics.dmetaphone("Smyth"))
```

```
('SM0', 'XMT') ('SM0', 'XMT')
```

Word Embeddings in NLP

- Word Embeddings are numeric representations of words in a lower-dimensional space, that capture semantic and syntactic information.
- They play a important role in Natural Language Processing (NLP) tasks.

Term frequency-inverse document frequency (TF-IDF)

- Term frequency-inverse document frequency is the machine learning algorithm that is used for **word embedding for text**. It comprises two metrics, namely term frequency (TF) and inverse document frequency (IDF).
- The term frequency (TF) score measures the frequency of words in a particular document. In simple words, it means that the occurrence of words is counted in the documents.
- The inverse document frequency or the IDF score measures the rarity of the words in the text. It is given more importance over the term frequency score because even though the TF score gives more weightage to frequently occurring words, the IDF score focuses on rarely used words in the corpus that may hold significant information.
- TF-IDF algorithm finds application in solving simpler natural language processing and machine learning problems for tasks like information retrieval, stop words removal, keyword extraction, and basic text analysis. However, it does not capture the semantic meaning of words efficiently in a sequence.

https://www.turing.com/kb/guide-on-word-embeddings-in-nlp

Term frequency-inverse document frequency (TF-IDF)-2

$$\textbf{tf-idf}_{i,j} = \text{Term Frequency}_{i,j} \times \text{Inverse Document Frequency}_i$$

Where,

$$\textbf{Term Frequency}_{i,j} = \frac{\text{Term i frequency in document j}}{\text{Total no. of terms in document j}}$$

$$\textbf{Inverse Document Frequency}_i = \log\left(\frac{\text{Total documents}}{\text{No. of documents containing term i}}\right)$$

TF-IDF example

Suppose we are looking for documents using the query *Q* and our database is composed of the documents *D1*, *D2,* and *D3*.

- *Q*: The cat.
- *D1*: The cat is on the mat.
- *D2*: My dog and cat are the best.
- *D3*: The locals are playing.

Step 1: Let's compute the TF scores of the words "the" and "cat" (i.e. the query words) with respect to the documents *D1*, *D2,* and *D3*.

TF("the", D1) = 2/6 = 0.33

TF("the", D2) = 1/7 = 0.14

TF("the", D3) = 1/4 = 0.25

TF("cat", D1) = 1/6 = 0.17

TF("cat", D2) = 1/7 = 0.14

TF("cat", D3) = 0/4 = 0

https://medium.com/nlplanet/two-minutes-nlp-learn-tf-idf-with-easy-examples-7c15957b4cb3

TF-IDF example-2

Step 2: IDF can be calculated by taking the total number of documents, dividing it by the number of documents that contain a word, and calculating the logarithm. If the word is very common and appears in many documents, this number will approach 0. Otherwise, it will approach 1.

IDF(word) = log(number of documents / number of documents that contain the word)

Let’s compute the IDF scores of the words “the” and “cat”.

IDF(“the”) = log(3/3) = log(1) = 0

IDF(“cat”) = log(3/2) = 0.18

https://medium.com/nlplanet/two-minutes-nlp-learn-tf-idf-with-easy-examples-7c15957b4cb3

TF-IDF example-3

Step 3: Multiplying TF and IDF gives the TF-IDF score of a word in a document. The higher the score, the more relevant that word is in that particular document.

TF-IDF(word, document) = TF(word, document) * IDF(word)

Let’s compute the TF-IDF scores of the words “the” and “cat”.

TF-IDF(“the”, D1) = 0.33 * 0 = 0

TF-IDF(“the, D2) = 0.14 * 0 = 0

TF-IDF(“the”, D3) = 0.25 * 0 = 0

TF-IDF(“cat”, D1) = 0.17 * 0.18= 0.0306

TF-IDF(“cat, D2) = 0.14 * 0.18= 0.0252

TF-IDF(“cat”, D3) = 0 * 0 = 0

https://medium.com/nlplanet/two-minutes-nlp-learn-tf-idf-with-easy-examples-7c15957b4cb3

TF-IDF example-4

Step4: The next step is to use a ranking function to order the documents according to the TF-IDF scores of their words. We can use the average TF-IDF word scores over each document to get the ranking of *D1*, *D2,* and *D3* with respect to the query *Q*.

Average TF-IDF of D1 = (0 + 0.0306) / 2 = 0.0153

Average TF-IDF of D2 = (0 + 0.0252) / 2 = 0.0126

Average TF-IDF of D3 = (0 + 0) / 2 = 0

Looks like the word “the” does not contribute to the TF-IDF scores of each document. This is because “the” appears in all of the documents and thus it is considered a not-relevant word.

As a conclusion, when performing the query “The cat” over the collection of documents *D1*, *D2,* and *D3*, the ranked results would be:

1. *D1*: The cat is on the mat.
2. *D2*: My dog and cat are the best.
3. *D3*: The locals are playing.

https://medium.com/nlplanet/two-minutes-nlp-learn-tf-idf-with-easy-examples-7c15957b4cb3

Word Embeddings

A word embedding is a mathematical representation that maps words to real-valued (numbers) vectors:

$$\text{word} \rightarrow R^n$$

where n represents the dimensionality of the embedding space.

The fundamental goal is to position semantically similar words close together in this vector space, enabling mathematical operations to capture linguistic relationships.

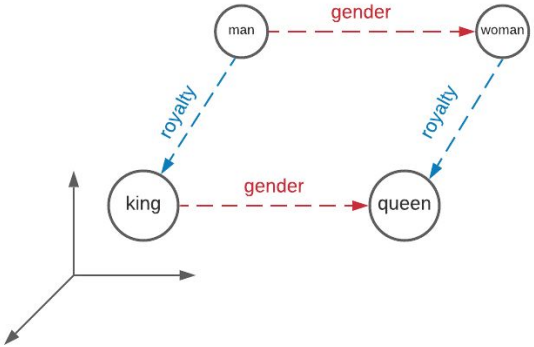
This dense representation typically uses hundreds of dimensions rather than the millions required by one-hot encoding approaches.

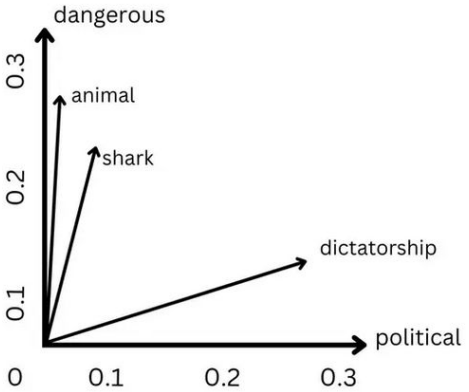
https://hunterheidenreich.com/posts/intro-to-word-embeddings/

Word Embeddings-2

the principle that words appearing in similar contexts tend to have similar meanings.

This forms the theoretical foundation of distributional semantics.





https://hunterheidenreich.com/posts/intro-to-word-embeddings/

Why Word Embeddings Matter in NLP

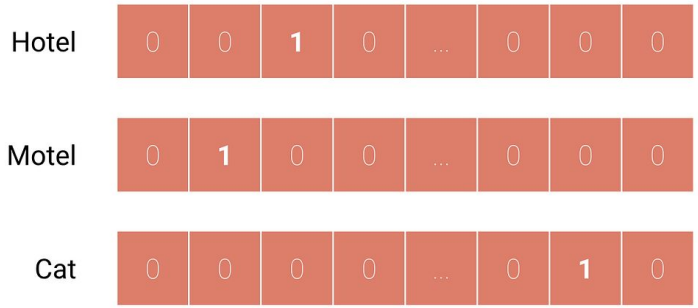
Computers cannot directly process words in their textual form—they require numerical representations to perform mathematical operations and apply machine learning algorithms. Word embeddings bridge this gap by converting linguistic tokens into dense vector representations that preserve semantic and syntactic relationships.

Key advantages:

1. **Dense representation**: Hundreds of dimensions instead of vocabulary-sized sparse vectors
2. **Semantic preservation**: Similar words cluster together in vector space
3. **Mathematical operations**: Enable analogical reasoning (e.g., king - man + woman ≈ queen)
4. **Transfer learning**: Pre-trained embeddings work across multiple tasks and domains

1-Sparse Representations

A simple way to represent words is through one-hot representations (a one-hot representation of a word is a N-dimensional vector with only one non-zero position corresponding to the index of that word). Let's index the vocabulary V by the set {1, …, N}.



Viewing all the words as *discrete units* is not ideal and the first problem that we would face when using one-hot representations is a sparsity problem.

In fact, since there can be hundreds of thousands of words in a given language, representing and storing words as one-hots can be extremely expensive.

https://blog.dataiku.com/arithmetic-properties-of-word-embeddings

2-Dense Representations

In 2013, Tomas Mikolov et al. developed an algorithm for learning word embeddings called Word2vec [paper][code].

This algorithm uses a *shallow neural network* to learn word vectors so that each word of a given corpus is good at predicting its own contexts (Skip-Gram) or vice versa (CBOW).

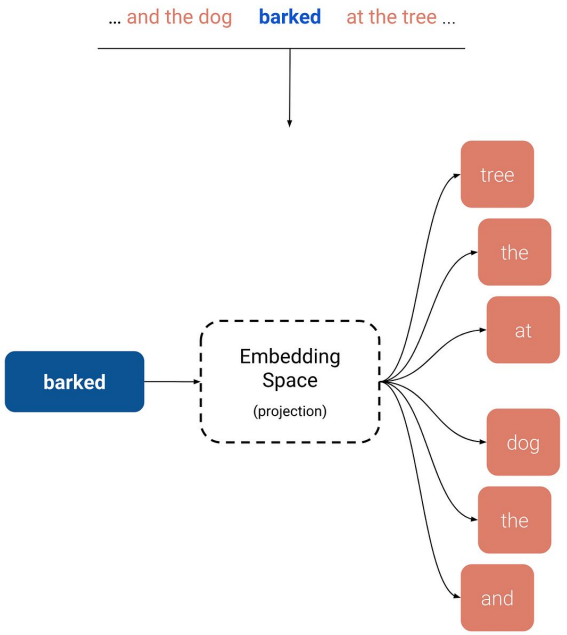
arithmetic of word vectors

For example, we can retrieve words that have *semantic* relations such as Country-Capital: عاصمة المغرب

```
vector(Paris) − vector(France) + vector(Morocco) ⇒ vector(Rabat)
```

As well as words that have *syntactic* relationships such as the Singular-Plural:

```
vector(Kings) - vector(King) + vector(Person) ⇒ vector(People)
```



Popular Word Embedding Techniques

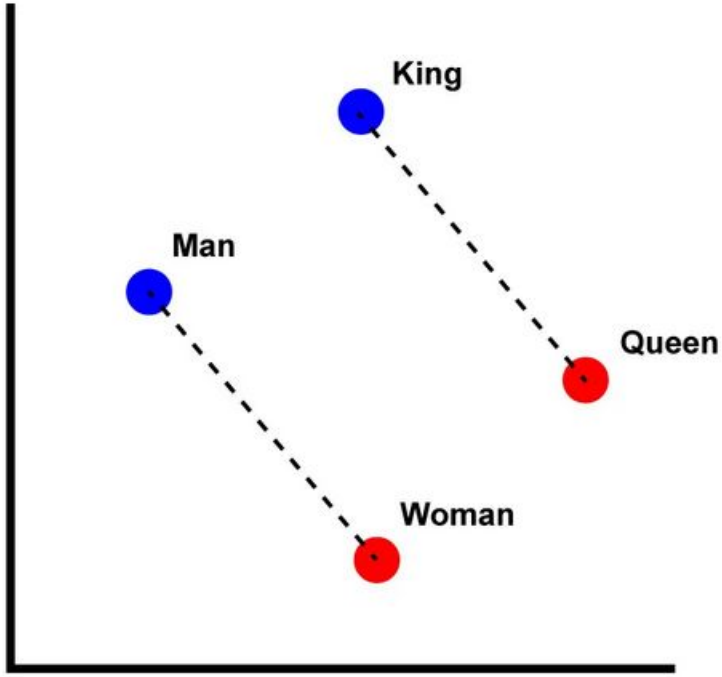
- **Word2Vec**: Explain Skip-gram and CBOW models.
- **GloVe**: Describe how it captures word co-occurrence statistics.
- **FastText**: Explain its subword-based approach for better handling of rare words.
- **Transformer-based Embeddings**: Briefly mention contextual embeddings like BERT and GPT.

1-Word2Vec

Word2Vec revolutionized word embeddings by introducing efficient training algorithms that could process massive corpora.

It became the first method to demonstrate compelling vector arithmetic properties, enabling analogical reasoning like the famous

“king - man + woman ≈ queen” example.



Word2Vec-2

Word2Vec uses **two architectures** to learn word representations:

1. **Continuous Bag of Words (CBOW)** — Predicts the target word from surrounding context words.
2. **Skip-Gram Model** — Predicts surrounding context words from a given word.

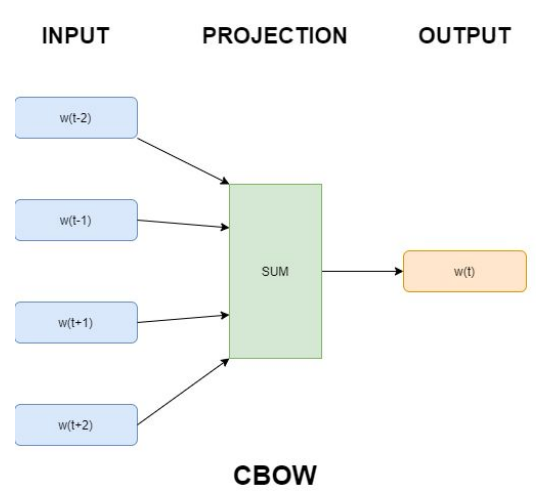
Key Advantage:

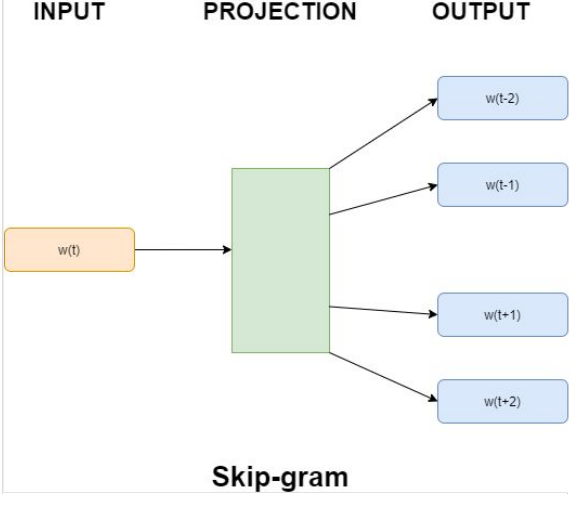
- Captures word similarities based on their usage in sentences.

Example: If trained on a large corpus, **“king”** and **“queen”** will have similar vector representations.

Key advantages:

- **Computational efficiency**: Much faster than neural probabilistic models
- **Scalable training**: Can process billion-word corpora effectively
- **Quality embeddings**: Captures semantic and syntactic relationships
- **Flexible context**: Window size controls topical vs. functional similarity





2-GloVe (Global Vectors for Word Representation, 2014)

GloVe is a word embedding model developed by **Stanford NLP researchers**. Unlike Word2Vec, which predicts words based on local context, GloVe focuses on **global co-occurrence statistics** of words.

Key Advantage:

- More effective for capturing long-range dependencies.
- Learns word relationships based on how often words appear together in a corpus.

Understanding Glove Data

Glove has pre-defined dense vectors for around every 6 billion words of English literature along with many other characters like commas, braces and semicolons. It can be downloaded and used immediately in many natural language processing (NLP) applications. Users can select a pre-trained GloVe embedding in a dimension like 50d, 100d, 200d or 300d vectors that best fits their needs in terms of computational resources and task specificity.

Here "d " stands for dimension. 100d means in this file each word has an equivalent vector of size 100.

https://www.geeksforgeeks.org/nlp/Glove-Word-Embedding-in-NLP/

How GloVe works ?

The GloVe algorithm works using the following process:

1. Preprocess the Text

First, we split the text into individual words (tokenization) so that we can work with them.

Example:

> Input text: "The peon is ringing the bell"
> Tokenized words: ['The', 'peon', 'is', 'ringing', 'the', 'bell']

2. Creating the Vocabulary

After tokenization, we create a list of all unique words in the text and then count how often each word appears.

Example:

> Vocabulary with word frequencies:
> {'The': 2, 'peon': 1, 'is': 1, 'ringing': 1, 'the': 1, 'bell': 1}

After this, the words are typically sorted by frequency.

How GloVe works ?-2

3. Building a Co-occurrence Matrix:

Now, we build a co-occurrence matrix where we count how often each word appears near other words in a given context (usually within a window of fixed size around the word).

Example: Let's say we choose a window size of 2 (2 words before and after each word). The co-occurrence matrix might look something like the matrix on the right side of the slide>>

In this matrix, the value at (i, j) represents how often word i and word j appear together in the context window.

	The	peon	is	ringing	the	bell
The	0	1	1	1	1	0
peon	1	0	1	1	0	0
is	1	1	0	1	1	0
ringing	1	1	1	0	1	1
the	1	0	1	1	0	1
bell	0	0	0	1	1	0

How GloVe works ?-3

4. Performing Dot Product

The aim is to learn word vectors such that the dot product of two word vectors reflects how often the words co-occur in the context. This ensures that words that appear in similar contexts will have similar vector representations.

Example:

"The" and "is" are frequently seen together, so their vectors will be close in the embedding space.

"peon" and "bell" don't co-occur much, so their vectors will be far apart.

5. Training the Word Vectors

Now, the model optimizes the word vectors by minimizing the difference between the dot product of word vectors and the expected co-occurrence probabilities (calculated as Pointwise Mutual Information, PMI). The goal is to adjust the vectors so that they correctly reflect the relationship between words based on the co-occurrence matrix.

Example:

"The" and "is" will have vector adjustments that make their dot product similar to their co-occurrence probability, ensuring their vectors are close to each other.

"peon" and "bell" will be adjusted to have distant vectors since their co-occurrence is low.

How GloVe works ?-4

6. Embedding Matrix

After training, the model outputs an embedding matrix where each word is represented by a dense vector. These vectors are able to capture the semantic and syntactic relationships between words.

Example: The resulting word vectors in the embedding matrix might look like this:

Word	Vector
The	[0.3, 0.1, 0.5]
peon	[0.2, 0.4, 0.3]
is	[0.6, 0.3, 0.4]
ringing	[0.1, 0.8, 0.7]
the	[0.3, 0.1, 0.5]
bell	[0.2, 0.3, 0.1]

Example code in python
(https://www.geeksforgeeks.org/nlp/Glove-Word-Embedding-in-NLP/)

3-FastText (Facebook AI, 2016)

FastText improves Word2Vec by using **subword information**. Instead of treating a word as a whole, it breaks it down into **n-grams (subword units)**.

Example: The word “apple” might be broken into subwords like **“app”**, **“ple”**, and **“le”**.

Key Advantage:

- Handles **rare words and misspellings** better than Word2Vec and GloVe.
- Useful for morphologically rich languages like German, Finnish, and Turkish.

4-Contextual Embeddings (BERT, GPT, etc.)

The other Traditional embeddings like Word2Vec, GloVe, and FastText generate **static** embeddings, meaning a word has the same vector regardless of its context.

Example: The word “bank” has the same embedding in:

1. “I deposited money in the **bank**.”
2. “The river **bank** was flooded.”

However, models like **BERT (Bidirectional Encoder Representations from Transformers)** and **GPT (Generative Pre-trained Transformer)** generate **contextual** embeddings, meaning the vector representation **changes based on the sentence’s meaning**. المعنى متغير حسب السياق

Key Advantage:

- Captures polysemy (multiple meanings of a word).
- Significantly improves performance in NLP tasks.

References

1. https://www.turing.com/kb/guide-on-word-embeddings-in-nlp ,
2. https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/
3. https://dataaspirant.com/five-most-popular-similarity-measures-implementation-in-python/
4. https://medium.com/@priadi_77114/similar-name-search-with-soundex-7f08d3fc93cb